

An Empirical Analysis of Fair Algorithms for Multi-Agent Multi-Armed Bandits

Arjun Chandra

April 2025

1 Introduction

The standard multi-armed bandit (MAB) problem involves a single agent that has access to a set of K arms. At each round, the agent selects an arm j which generates a stochastic reward for the agent from an unknown distribution D_j with an unknown mean μ_j^* . The goal of the agent is to identify the best arm as fast as possible in order to minimize the agent's cumulative regret up to T rounds, defined as $\sum_{t=1}^T (\max_j \mu_j^* - \mu_{a_t}^*)$. This problem setup can be used to model a wide variety of real-life scenarios including clinical trials, online recommendations, and A/B testing. The common theme among these applications is that a single agent must determine the best action to take to maximize their individual reward. However, there are many real-life scenarios in which taking an action affects not one, but multiple agents. For example, a city may be evaluating alternatives to its current transportation program which would affect all of the the city's public transportation users (i.e., the agents). In such scenarios, the decision-making entity must take into account the preferences of all of the agents to determine how to take actions in a way that is *fair* to the agents. This idea has been formalized as a variant of a multi-agent multi-armed bandit (MA-MAB) problem, and it is of clear relevance to many policy decision scenarios. This report aims to provide an overview of fair algorithms in the MA-MAB setting proposed by Hossain et al. [5] and empirically analyze the performance and practicality of these algorithms. The implementation is available at <https://github.com/arjunchandra2/DS592-Final-Project>.

2 Background

Multi-agent multi-armed bandit (MA-MAB) problems have been studied in a variety of settings [1, 4, 5, 6, 7]. Agarwal et al. [1] study an MA-MAB problem where N agents collaborate on a shared instance of an MAB problem where communication between agents is limited to certain rounds and by the number of bits in each communication round. Other prior works [4, 7] propose their own variants of the collaborative MA-MAB setting and develop decentralized algorithms for these variants. In the realm of fair decision making, Hossain et al. [5] study an MA-MAB variant in which a single decision-making entity must take into account the preferences of N agents in order to learn a fair policy over K arms by maximizing a fairness metric called the Nash Social Welfare (NSW). This MA-MAB variant is of significant interest as it is applicable to many real-life policy decision scenarios, and as such it will serve as the focus for the rest of this report. The paper proposes three novel algorithms for their MA-MAB variant by introducing natural modifications to Explore-Then-Commit, Epsilon-Greedy, and UCB. They derive sublinear regret bounds which are similar to the classic MAB counterparts of each algorithm; however, their regret analysis is complicated by the NSW objective and requires new mathematical techniques such as a Lipschitz-continuity analysis. This report will provide an overview and proof sketches for each of the algorithms and main regret bounds derived in [5]. We also present a simplified version of the proof for the Epsilon-Greedy regret bound in Section 4.2. Section 5 then presents experimental results for each algorithm, which were not included in the original paper. Jones et al. [6] later revisit the MA-MAB problem proposed in [5] and introduce a modified UCB algorithm that

offers a more practical implementation. Indeed, the experiments in Section 5 will also highlight the practical issues of the UCB-variant proposed in [5] and motivate the algorithms introduced by [6].

3 Problem Setup

3.1 Preliminaries

Notation. For $n \in \mathbb{N}$, define $[n] = \{1, \dots, n\}$. Let $N, K \in \mathbb{N}$. In this MA-MAB problem, there is a set of agents $[N]$ and a set of arms $[K]$. For each agent $i \in [N]$ and arm $j \in [K]$, there is a reward distribution $D_{i,j}$ with mean $\mu_{i,j}^*$ and support $[0, 1]$. When arm j is pulled, each agent i observes an independent reward sampled from $D_{i,j}$. The true reward matrix is denoted as $\mu^* = (\mu_{i,j}^*)_{i \in [N], j \in [K]} \in [0, 1]^{N \times K}$.

Policies. We are interested in learning probability distributions over arms, referred to as a policy. The K -simplex, denoted Δ^K , is the set of all policies. For a policy $p \in \Delta^K$, p_j denotes the probability with which arm j is pulled.

Nash social welfare. The Nash social welfare (NSW) is defined as the product of expected rewards to the agents. Given $\mu = (\mu_{i,j})_{i \in [N], j \in [K]}$, and policy $p \in \Delta^K$, the NSW is defined as:

$$\text{NSW}(p, \mu) = \prod_{i=1}^N \left(\sum_{j=1}^K p_j \cdot \mu_{i,j} \right).$$

The true Nash social welfare under policy p is $\text{NSW}(p, \mu^*)$. If we knew μ^* , we would pick an optimal policy $p^* \in \arg \max_{p \in \Delta^K} \text{NSW}(p, \mu^*)$. However, because we do not know μ^* in advance, the algorithms described in Section 4 will produce an estimate $\hat{\mu}$, and use it to choose a policy.

Algorithms. An algorithm for the MA-MAB problem chooses a policy p^t in each round $t \in \mathbb{N}$. Then, an arm a^t is sampled according to policy p^t , and for each agent $i \in [N]$, a reward X_{i,a^t}^t is sampled independently from distribution D_{i,a^t} . The algorithm observes the reward vector $(X_{i,a^t}^t)_{i \in [N]}$, which it can use to choose policies in the later rounds.

Reward estimates. All the algorithms maintain an estimate of the mean reward matrix μ^* at every round. For round t and arm $j \in [K]$, define:

$$n_j^t = \sum_{s=1}^{t-1} \mathbf{1}[a_s = j]$$

Evidently, n_j^t denotes the number of times arm j is pulled at the beginning of round t . Next we define:

$$\hat{\mu}_{i,j}^t = \frac{1}{n_j^t} \sum_{s \in [t-1]: a_s = j} X_{i,j}^s$$

$\hat{\mu}_{i,j}^t$ denotes the average reward experienced by agent i from the n_j^t pulls of arm j thus far. The algorithms treat these as an estimate of $\mu_{i,j}^*$ available at the beginning of round t . Let $\hat{\mu}^t = (\hat{\mu}_{i,j}^t)_{i \in [N], j \in [K]}$.

Regret. Recall that p^* is an optimal policy that has the highest Nash social welfare. The instantaneous regret in round t due to an algorithm choosing a policy p^t is:

$$r^t = \text{NSW}(p^*, \mu^*) - \text{NSW}(p^t, \mu^*)$$

The cumulative regret up to round T due to an algorithm choosing policies $p^1, \dots, p^T$ at each round is:

$$R^T = \sum_{t=1}^T r^t$$

We are interested in bounding the expected regret $\mathbb{E}[R^T]$ of an algorithm at round T , where the expectation is over the randomness involved in sampling the arms a^t and the agent rewards $(X_{i,a^t}^t)_{i \in [N]}$ for $t \in [T]$.

3.2 Nash Social Welfare

It is not immediately obvious why the Nash social welfare is an appropriate fairness metric for this problem setup. A brief explanation is given in [5]; however, we provide further justification by illustrating motivating examples of NSW in the case of 3 agents and 3 arms ($N = 3, K = 3$) in Fig. 1 below.

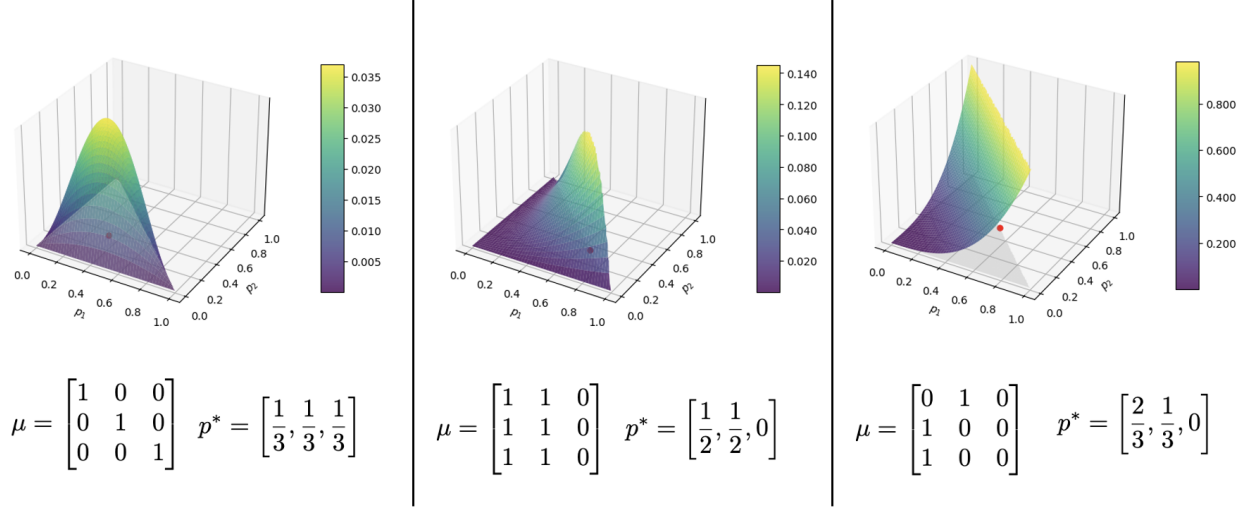


Figure 1: Nash social welfare yields fair policies in intuitive scenarios. The optimal policy is indicated by the red point. **Left:** All three agents prefer different arms. The optimal policy assigns equal probability to all three arms. **Middle:** All three agents prefer the first two arms equally and none prefer the third. The optimal policy assigns equal probability to the first two arms and zero to the third. **Right:** Two agents prefer the first arm and the third agent prefers the second arm. The optimal policy assigns probabilities proportional to the number of agents that prefer each arm.

Note that the key notion of fairness which is captured by NSW is that the optimal policy should assign probabilities to each arm proportional to the number of agents that prefer that arm. This should be a very intuitive notion of fairness. For example, imagine that a city with 100 residents is choosing between two possible transportation schedules. If 60 agents prefer the first schedule and 40 agents prefer the second schedule, the NSW dictates that the city should choose the first schedule with $p_1 = 0.6$ and the second schedule with $p_2 = 0.4$. Evidently the situation is less clear when the weights of each agent’s preferences differ, in which case the reader is referred to [2] for a more in-depth discussion. Finally, another desirable property of the NSW is that it is log-concave, meaning it can be maximized by standard convex programming tools [3]. This is a very useful fact for the implementation of the algorithms that are to follow.

4 Fair Algorithms

This section will present the Explore-Then-Commit, Epsilon-Greedy, and UCB variants introduced in [5]. We also include a brief proof sketch for each of the main regret bounds and a more involved proof sketch for Epsilon-Greedy in Section 4.2. For complete proofs, please refer to the original paper.

4.1 Explore-Then-Commit

The Explore-Then-Commit algorithm is shown below. The algorithm closely mirrors the logic of its MAB counterpart. It initially explores each arm L times to construct an estimator $\hat{\mu}$ of the true mean rewards μ . The algorithm then uses the estimator $\hat{\mu}$ constructed during exploration to compute a policy p that it exploits for the rest of the rounds.

ALGORITHM 1: Explore-Then-Commit

Input: Number of agents N , number of arms K , horizon T

Parameters : Exploration period L

// Pull each arm L times

for $t = 1, \dots, K \cdot L$ **do** // Exploration

$j \leftarrow \lceil t/L \rceil$

$p^t \leftarrow$ policy that puts probability 1 on arm j // Pull arm j deterministically

end

Compute the estimated reward matrix $\hat{\mu} \triangleq \hat{\mu}^{K \cdot L + 1}$ of the rewards observed so far

Compute $\hat{p} \in \arg \max_{p \in \Delta^K} \text{NSW}(p, \hat{\mu})$

for $t = K \cdot L + 1, \dots, T$ **do** // Exploitation

$p^t \leftarrow \hat{p}$

end

Theorem 1. *Explore-Then-Commit achieves a regret bound $\mathbb{E}[R^T] = \mathcal{O}\left(N^{\frac{2}{3}} K^{\frac{1}{3}} T^{\frac{2}{3}} \log^{\frac{1}{3}}(NKT)\right)$ when $L = \Theta\left(N^{\frac{2}{3}} K^{-\frac{2}{3}} T^{\frac{2}{3}} \log^{\frac{1}{3}}(NKT)\right)$.*

Proof Sketch. The proof is very similar to the regret analysis for the classic Explore-Then-Commit MAB algorithm. We start by splitting the regret into two terms: one for the regret incurred during the exploration phase and one for the regret incurred in the exploitation phase. We also use that the instantaneous regret at each round can be at most 1 since $\text{NSW}(p, \mu^*) \in [0, 1]$ for every policy p :

$$\mathbb{E}[R^T] = \sum_{t=1}^T \mathbb{E}[r^t] \leq KL \cdot 1 + (T - KL) \cdot \mathbb{E}[\text{NSW}(p^*, \mu^*) - \text{NSW}(\hat{p}, \mu^*)]. \quad (1)$$

Now we must bound $\mathbb{E}[\text{NSW}(p^*, \mu^*) - \text{NSW}(\hat{p}, \mu^*)]$. To do so, we define the good event for the exploitation phase:

$$\mathcal{E}_{\text{good}} \triangleq \{\forall i \in [N], \forall j \in [K] : |\hat{\mu}_{i,j} - \mu_{i,j}^*| \leq \epsilon\}$$

Intuitively, when the good event holds, our estimated reward matrix is not too far from the true reward matrix. Note that here the good event does not have a dependence on t because the estimated reward matrix never changes during the exploitation phase. Then we specify $\epsilon = \sqrt{\frac{\log(NKT)}{L}}$ and show that $\Pr[\mathcal{E}_{\text{good}}] \geq 1 - 2/T^2$. Before we condition on the good event, we also introduce the following lemma about the Lipschitz-continuity of $\text{NSW}(p, \mu)$:

Lemma 1. *Given a policy $p \in \Delta^K$, and reward matrices $\mu^1, \mu^2 \in [0, 1]^{N \times K}$, we have*

$$|\text{NSW}(p, \mu^1) - \text{NSW}(p, \mu^2)| \leq \sum_{i \in [N]} \sum_{j \in [K]} p_j \cdot |\mu_{i,j}^1 - \mu_{i,j}^2|.$$

Using Lemma 1 and the fact that the algorithm chooses $\hat{p} \in \arg \max_{p \in \Delta^K} \text{NSW}(p, \hat{\mu})$, we can condition on $\mathcal{E}_{\text{good}}$ and show that:

$$\text{NSW}(p^*, \mu^*) \leq \text{NSW}(\hat{p}, \mu^*) + 2N\epsilon,$$

We substitute this bound into the second term of Equation 1 conditioned on the good event — $\mathbb{E}[\text{NSW}(p^*, \mu^*) - \text{NSW}(\hat{p}, \mu^*)] \leq \Pr[\mathcal{E}] \cdot \mathbb{E}[\text{NSW}(p^*, \mu^*) - \text{NSW}(\hat{p}, \mu^*) | \mathcal{E}] + \Pr[\neg \mathcal{E}] \cdot 1$. Setting $L = \Theta\left(N^{\frac{2}{3}} K^{-\frac{2}{3}} T^{\frac{2}{3}} \log^{\frac{1}{3}}(NKT)\right)$ completes the proof. $\square$

4.2 Epsilon-Greedy

The Epsilon-Greedy algorithm is shown below. Once again, the algorithm closely mirrors the logic of its MAB counterpart. At each round t , with probability ϵ^t , the algorithm selects an arm using round-robin. Otherwise, the algorithm uses its estimator $\hat{\mu}$ constructed using the previous observed rewards to compute the policy for round t .

ALGORITHM 2: ϵ^t -Greedy

Input: Number of agents N , number of arms K

Parameters : Exploration probabilities ϵ^t for $t \in \mathbb{N}$

```

curr ← 1 // Next arm to pull during exploration
for t = 1, 2, ..., do
    Toss a coin with success probability  $\epsilon^t$ 
    if success then // Exploration
        // Round-robin among arms during exploration
         $p^t \leftarrow$  policy that puts probability 1 on arm curr // Pull it deterministically
        curr ← curr + 1 // When curr becomes K + 1, reset to 1
    else // Exploitation
        Compute the estimated reward matrix  $\hat{\mu}^t$  from the rewards observed so far
         $p^t \leftarrow \arg \max_{p \in \Delta^K} \text{NSW}(p, \hat{\mu}^t)$ 
    end
end
end

```

Theorem 2. *Epsilon-Greedy achieves a regret bound $\mathbb{E}[R^T] = \mathcal{O}\left(N^{\frac{2}{3}} K^{\frac{1}{3}} T^{\frac{2}{3}} \log^{\frac{1}{3}}(NKT)\right)$ when $\epsilon^t = \Theta\left(N^{\frac{2}{3}} K^{\frac{1}{3}} t^{-\frac{1}{3}} \log^{\frac{1}{3}}(NKT)\right)$.*

Proof Sketch. The original paper provides two regret bounds for Epsilon-Greedy for two different choices of ϵ^t . However, in this report we will only prove the regret bound for the choice of ϵ^t given in Theorem 2 as it yields a simpler proof. We start as usual by splitting the regret into two terms: one for the regret incurred during exploration rounds and one for the regret incurred during exploitation rounds. We also use that the instantaneous regret at each round can be at most 1 since $\text{NSW}(p, \mu^*) \in [0, 1]$ for every policy p :

$$\mathbb{E}[R^T] = \sum_{t=1}^T \mathbb{E}[r^t] \leq \sum_{t=1}^T \mathbb{E}\left[\epsilon^t \cdot 1 + (1 - \epsilon^t) \cdot (\text{NSW}(p^*, \mu^*) - \text{NSW}(\hat{p}^t, \mu^*))\right] \quad (2)$$

We turn our attention to bounding $\text{NSW}(p^*, \mu^*) - \text{NSW}(\hat{p}^t, \mu^*)$. To do so, we introduce the following lemmas:

Lemma 2. Define $r_j^t = \sqrt{\frac{2\log(NKt)}{n_j^t}}$, and event

$$\mathcal{E}^t \triangleq \forall i \in [N], j \in [K] : |\hat{\mu}_{i,j}^t - \mu_{i,j}^*| \leq r_j^t.$$

Then, for any algorithm and any t , we have $\Pr[\mathcal{E}^t] \geq 1 - \frac{2}{t^3}$.

Lemma 3. Conditioned on $\mathcal{E}^t$, we have that

$$\forall p \in \Delta^K : |\text{NSW}(p, \hat{\mu}^t) - \text{NSW}(p, \mu^*)| \leq N \cdot \sum_{j \in [K]} p_j \cdot r_j^t$$

Lemma 2 defines the event where the algorithm's estimated reward matrix at round t is close to the true reward matrix and shows that it holds with high probability. Lemma 3 provides a bound which follows from a direct application of Lemma 1 conditioned on the event $\mathcal{E}^t$. We then introduce a second event:

$$\mathcal{B}^t \triangleq b^t \geq \gamma \cdot t\epsilon^t$$

b^t denotes the number of times Epsilon-Greedy performs exploration up to round t , $\gamma = 1 - 1/\theta$, and $\theta > 0$ is a constant such that $\epsilon^t \geq \theta \cdot t^{-1/3}$. Then, by Hoeffding's inequality, we have:

$$\Pr[-\mathcal{B}^t] \leq e^{-2(1-\gamma)^2\theta^2 t^{1/3}} = e^{-2t^{1/3}} \leq e^{-\log t} = \frac{1}{t}. \quad (3)$$

Because the algorithm performs round-robin during exploration, conditioned on $\mathcal{B}^t$, we have that $n_j^t \geq \frac{b^t}{K} \geq \frac{\gamma \cdot t\epsilon^t}{K}$ for each arm j , which implies $r_j^t \leq \sqrt{\frac{2K\log(NKt)}{\gamma \cdot t\epsilon^t}}$ for each j . Thus, conditioned on $\mathcal{B}^t$, we have:

$$\forall p \in \Delta^K : \sum_{j \in [K]} p_j \cdot r_j^t \leq \max_{j \in [K]} r_j^t \leq \sqrt{\frac{2K\log(NKt)}{\gamma \cdot t\epsilon^t}}. \quad (4)$$

We can now bound $\text{NSW}(p^*, \mu^*) - \text{NSW}(\hat{p}, \mu^*)$ conditioned on $\mathcal{E}^t \wedge \mathcal{B}^t$:

$$\begin{aligned} & \text{NSW}(p^*, \mu^*) - \text{NSW}(\hat{p}^t, \mu^*) \\ &= (\text{NSW}(p^*, \mu^*) - \text{NSW}(p^*, \hat{\mu}^t)) + (\text{NSW}(p^*, \hat{\mu}^t) - \text{NSW}(\hat{p}^t, \hat{\mu}^t)) + (\text{NSW}(\hat{p}^t, \hat{\mu}^t) - \text{NSW}(\hat{p}^t, \mu^*)) \\ &\leq \sum_{p \in \{p^*, \hat{p}^t\}} |\text{NSW}(p, \mu^*) - \text{NSW}(p, \hat{\mu}^t)| \leq 2N \sqrt{\frac{2K\log(NKt)}{\gamma \cdot t\epsilon^t}} \end{aligned} \quad (5)$$

The last transition holds using Equation 4 and because $\hat{p}^t$ is the optimal policy under $\hat{\mu}^t$, so $\text{NSW}(p^*, \hat{\mu}^t) \leq \text{NSW}(\hat{p}^t, \hat{\mu}^t)$. We can now analyze the expected regret of Epsilon-Greedy at round T . We have:

$$\begin{aligned} \mathbb{E}[R^T] &= \sum_{t=1}^T \mathbb{E}[r^t] \leq \sum_{t=1}^T \mathbb{E}[\epsilon^t \cdot 1 + (1 - \epsilon^t) \cdot (\text{NSW}(p^*, \mu^*) - \text{NSW}(\hat{p}^t, \mu^*))] \\ &\leq \sum_{t=1}^T \left(\epsilon^t + \Pr[\mathcal{E}^t \wedge \mathcal{B}^t] \cdot \mathbb{E}[\text{NSW}(p^*, \mu^*) - \text{NSW}(\hat{p}^t, \mu^*) \mid \mathcal{E}^t \wedge \mathcal{B}^t] \right. \\ &\quad \left. + \Pr[-\mathcal{E}^t \vee -\mathcal{B}^t] \cdot 1 \right) \\ &\leq \sum_{t=1}^T \left(\epsilon^t + 2N \sqrt{\frac{2K\log(NKt)}{\gamma \cdot t\epsilon^t}} + \frac{2}{t^3} + \frac{1}{t} \right). \end{aligned}$$

In the last transition, we use Equation 5, we naively bound $\Pr[\mathcal{E}^t \wedge \mathcal{B}^t] \leq 1$, and we bound $\Pr[\neg \mathcal{E}^t \vee \neg \mathcal{B}^t] \leq \frac{2}{t^3} + \frac{1}{t}$ using Lemma 2, Equation 3, and the union bound. Setting $\epsilon^t = \Theta\left(N^{\frac{2}{3}} K^{\frac{1}{3}} t^{-\frac{1}{3}} \log^{\frac{1}{3}}(NKT)\right)$ completes the proof. $\square$

4.3 UCB

The UCB algorithm is shown below. The algorithm initially pulls each arm once to construct an estimated reward matrix $\hat{\mu}$. For each subsequent round t , the algorithm computes the policy using the current estimate of $\hat{\mu}$ with an added linear bonus term for each arm analogous to the bonus radius used in the classic UCB algorithm for MAB.

ALGORITHM 3: UCB

Input: Number of agents N , number of arms K

Parameters : Confidence parameter α^t for each $t \in \mathbb{N}$

// Pull each arm once

for $t = 1, \dots, K$ **do**

$p^t \leftarrow$ policy that puts probability 1 on arm t // Pull arm t deterministically

end

for $t = K + 1, \dots$ **do**

 Compute the estimated reward matrix $\hat{\mu}^t$

$p^t \leftarrow \arg \max_{p \in \Delta^K} \text{NSW}(p, \hat{\mu}^t) + \alpha^t \sum_{j \in [K]} p_j \cdot r_j^t$, where $r_j^t \triangleq \sqrt{\frac{\log(NKt)}{n_j^t}}$.

end

Theorem 3. *UCB achieves a regret bound $\mathbb{E}[R^T] = \mathcal{O}\left(NKT^{\frac{1}{2}} \log(NKT)\right)$ when $\alpha^t = N$ for all t .*

Proof Sketch. To prove the desired bound we first define $\alpha^t = N$ for all t and $c = N$. Then we define the following event:

$$\mathcal{C}_\alpha^t \triangleq \forall p \in \Delta^K : |\text{NSW}(p, \mu^*) - \text{NSW}(p, \hat{\mu}^t)| \leq \alpha^t \cdot \sum_{j \in [K]} p_j \cdot r_j^t + \frac{4}{t}.$$

Note that, as it's written, $\mathcal{C}_\alpha^t$ appears to be a looser bound for the same event we defined in Lemma 3. However, the original paper also shows that the event holds for $\alpha^t = \sqrt{12NK \log(NKt)}$ by introducing another event which we have omitted here, but we otherwise follow their proof. We can show that $\Pr[\neg \mathcal{C}_\alpha^t] \leq 4/t^3$. Then we define the good event $\mathcal{C}_\alpha^* \triangleq \bigwedge_{t \geq \sqrt{T}} \mathcal{C}_\alpha^t$ because we are interested in $\mathcal{C}_\alpha^t$ holding for $t \geq \sqrt{T}$. Then, for any $t \in [T]$, conditioned on $\mathcal{C}_\alpha^t$ we can show that:

$$\text{NSW}(p^*, \mu^*) \leq \text{NSW}(p^t, \mu^*) + 2\alpha^t \sum_{j \in [K]} p_j^t \cdot r_j^t + \frac{8}{t}$$

Before using the good event, we will introduce extra notation so that we can condition on a specific sequence of policies taken by the algorithm. Let us write $p^{[T]} = (p^1, \dots, p^T)$ for the random variable denoting the policies used by the algorithm, and $\bar{p}^{[T]} = (\bar{p}^1, \dots, \bar{p}^T)$ to denote a specific value in $(\Delta^K)^T$ taken by the random variable. Now fix an arbitrary $\bar{p}^{[T]}$. For $t \in [T]$ and $j \in [K]$, define $q_j^t = \sum_{s=1}^t \bar{p}_j^s$. Then, $\mathbb{E}[n_j^t | p^{[T]}] = \bar{p}_j^{[T]}$. For each $j \in [K]$, let T_j be the smallest t for which $q_j^t \geq 2\sqrt{T \log(NKT)}$. If no such t exists, let $T_j = T$. Then we define another event:

$$\mathcal{B} \triangleq \forall j \in [K], n_{j}^{T_j} \geq \sqrt{T \log(NKT)}$$

It can be shown that $\Pr[\neg \mathcal{B} \mid \mathcal{C}_\alpha^* \wedge p^{[T]} = \bar{p}^{[T]}] \leq \frac{1}{N^2 K T^2}$. Then we can bound $\mathbb{E}[R^T \mid \mathcal{C}_\alpha^* \wedge p^{[T]} = \bar{p}^{[T]}]$ by using event $\mathcal{B}$:

$$\mathbb{E}\left[R^T \mid \mathcal{C}_\alpha^* \wedge p^{[T]} = \bar{p}^{[T]}\right] \leq \max(K, \sqrt{T}) + 1 + 2c\sqrt{2\log(NKT)} \sum_{t=1}^T \sum_{j \in [K]} \frac{\bar{p}_j^t}{\sqrt{c_j}}$$

Where we have defined $c_j = 1$ if $t < T_j$, and $c_j = \sqrt{T \log(NKT)}$ if $t \geq T_j$. From this we can show:

$$\mathbb{E}\left[R^T \mid \mathcal{C}_\alpha^* \wedge p^{[T]} = \bar{p}^{[T]}\right] = \mathcal{O}\left(cK\sqrt{T} \log(NKT)\right)$$

The bound above holds for every possible $\bar{p}^{[T]}$, so we also have that $\mathbb{E}[R^T \mid \mathcal{C}_\alpha^*] = \mathcal{O}\left(cK\sqrt{T} \log(NKT)\right)$. Therefore, we bound the regret by conditioning on $\mathcal{C}_\alpha^*$:

$$\begin{aligned} \mathbb{E}[R^T] &= \Pr[\mathcal{C}_\alpha^*] \cdot \mathbb{E}[R^T \mid \mathcal{C}_\alpha^*] + \Pr[\neg \mathcal{C}_\alpha^*] \cdot \mathbb{E}[R^T \mid \neg \mathcal{C}_\alpha^*] \\ &\leq 1 \cdot \mathcal{O}\left(cK\sqrt{T} \log(NKT)\right) + \frac{4}{\sqrt{T}} \cdot 1 = \mathcal{O}\left(cK\sqrt{T} \log(NKT)\right). \end{aligned}$$

Substituting $c = N$ completes the proof. □

5 Experiments

In this section we experimentally validate the regret bounds for each algorithm introduced in Section 4.

5.1 Experimental Protocol

Unless otherwise specified, all of the experiments use $N = 3$ agents and $K = 3$ arms with the following reward matrix:

$$\mu^* = \begin{bmatrix} 0.3 & 0.5 & 0.7 \\ 0.5 & 0.7 & 0.3 \\ 0.7 & 0.3 & 0.5 \end{bmatrix}, \quad p^* = \left[\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right]$$

Each distribution is then $D_{i,j} = \mathcal{N}(\mu_{i,j}^*, 0.1)$. It ensures that $\Pr[X_{i,a_t}^t \in [0, 1]] \geq 0.997$ for all $i \in [N], t \in [T]$ as a heuristic for the assumption that each $D_{i,j}$ has support $[0, 1]$. Also, the only non-trivial step in the implementation for each algorithm is to compute $\arg \max_{p \in \Delta^K} \text{NSW}(p, \hat{\mu}^t)$ (plus the bonus term for UCB). For this optimization, we use the `SLSQP` optimizer from `scipy.optimize`. To enforce that $p \in \Delta^K$ we specify that each $p_i \in [0, 1]$ and $\sum_{i=1}^K p_i = 1$. The initial guess for the optimization is a policy that assigns uniform probability across the K arms.

5.2 Explore-Then-Commit

5.2.1 Regret Bound Validation

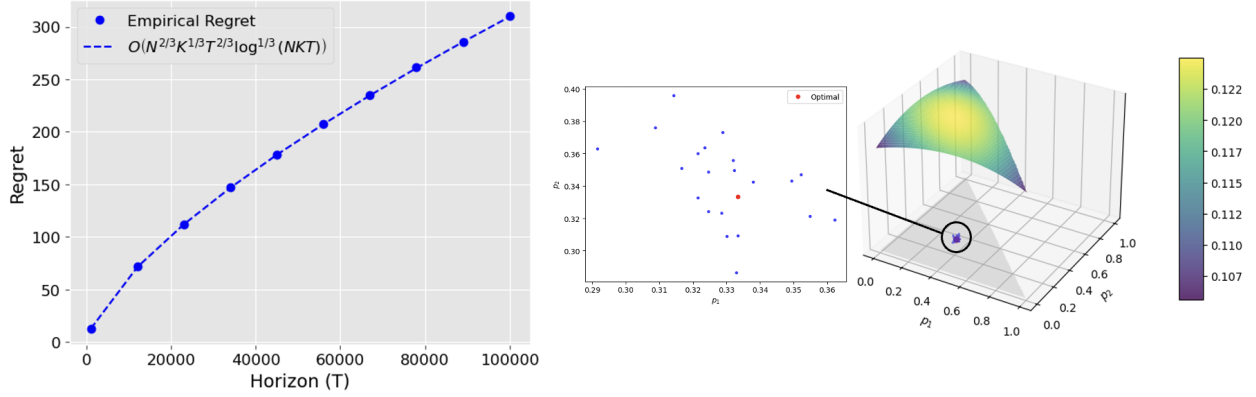


Figure 2: Explore-Then-Commit. **Left:** Expected Regret vs. Horizon. **Right:** NSW with μ^* , the optimal p^* , and the distribution of exploitation policies for $T = 10^4$.

Fig. 2 validates that the theoretical regret for Explore-Then-Commit matches the empirical regret. For this experiment, the regret is averaged across 10 simulations for each horizon as a proxy for the expected regret. On the right, we also show the distribution of exploitation policies (i.e., the policy computed after the exploration phase) for 30 simulations with $T = 10^4$ along with the NSW function for the true reward matrix μ^* . We observe that the policy used by each simulation for the exploitation rounds is close to the optimal policy.

5.2.2 Regret Bound Tradeoff

Hossain et al. [5] also derive an alternate regret bound from the one provided in Section 4.1 for a different choice of L . This tradeoff is illustrated in Fig. 3 (next page). The plot on the left shows the regret for $N = 2$ agents and $K = 100$ arms averaged over 5 simulations for each horizon. Each arm $j \in [K]$ has one agent with $\mu_{*,j}^* = 0.3$ and the other agent with $\mu_{*,j}^* = 0.5$. The plot on the right shows the regret for $N = 100$ agents and $K = 2$ arms averaged over 5 simulations for each horizon. In this setting, we simply transpose the reward matrix from the previous setting so that each of the two arms is preferred by one half of the agents.

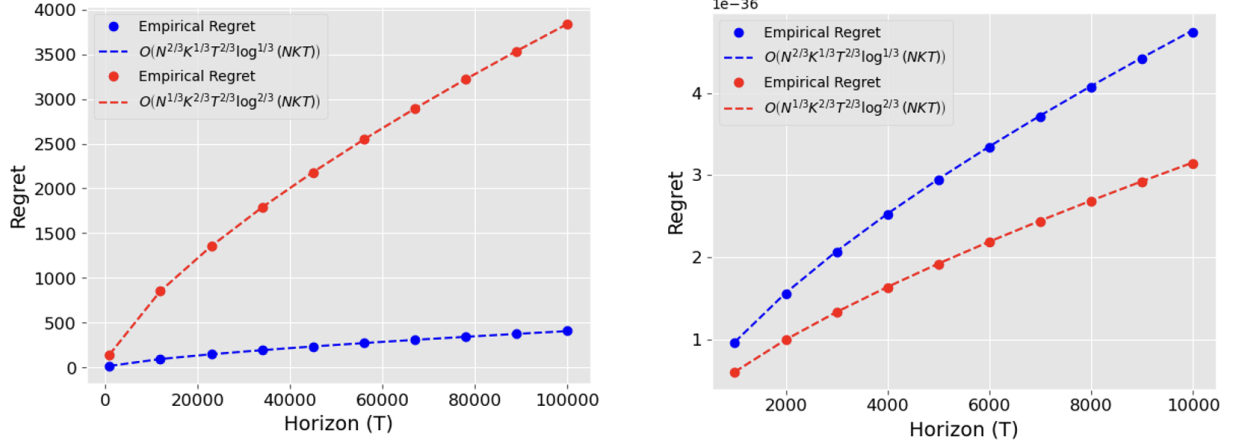


Figure 3: Explore-Then-Commit Regret Tradeoff. **Left:** Expected Regret vs. Horizon for $N = 2$ agents and $K = 100$ arms. **Right:** Expected Regret vs. Horizon for $N = 100$ agents and $K = 2$ arms.

5.3 Epsilon-Greedy

5.3.1 Regret Bound Validation

Fig. 4 validates that the theoretical regret bound for Epsilon-Greedy matches the empirical regret. For this experiment, the regret is averaged across 10 simulations for each horizon. On the right, we show the trajectories of the learned policies by sampling the policies at evenly spaced intervals during exploitation rounds for 3 simulations with $T = 10^4$. One particularly interesting result which is evident for the policy trajectory shown in green is that it is possible for the proposed Epsilon-Greedy algorithm to learn the optimal policy and then later deviate from it. That is, there is no guarantee that the algorithm will remain at the optimal solution once it is found. This would be a desirable property, and future work could aim to suggest modifications to the proposed Epsilon-Greedy to guarantee that it will converge and remain at the optimal solution.

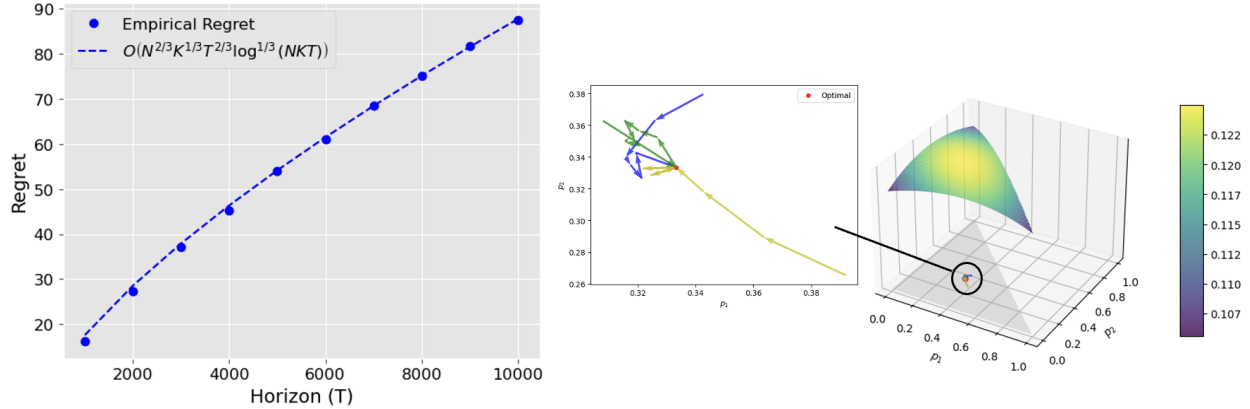


Figure 4: Epsilon-Greedy. **Left:** Expected Regret vs. Horizon. **Right:** NSW with μ^* , the optimal p^* , and the policy trajectories for $T = 10^4$.

5.4 UCB

5.4.1 Regret Bound Validation

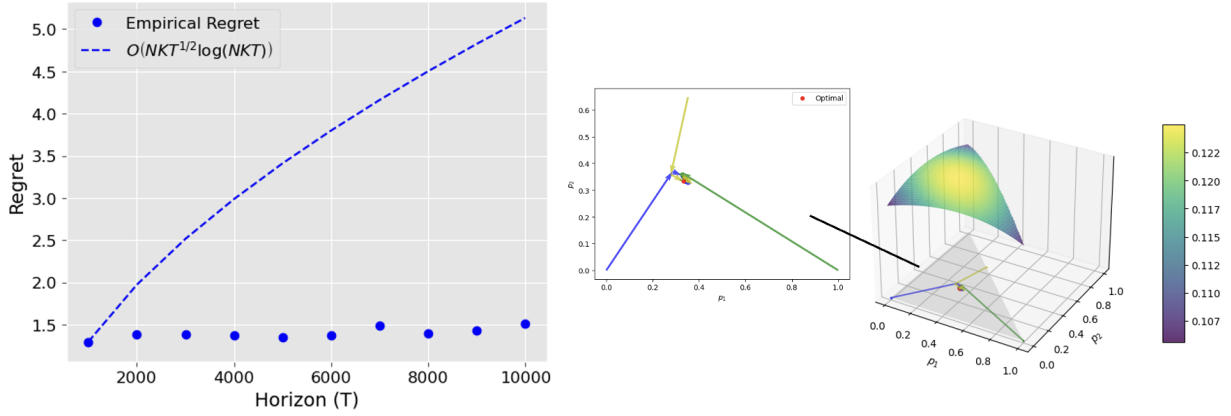


Figure 5: UCB. **Left:** Expected Regret vs. Horizon. **Right:** NSW with μ^* , the optimal p^* , and the policy trajectories for $T = 10^4$.

Fig. 5 validates that the theoretical regret bound for UCB matches the empirical regret. For this experiment, the regret is averaged across 10 simulations for each horizon. We also observe that the regret of UCB is relatively constant for different horizons. On the right, we show the trajectories of the learned policies by sampling the policies at evenly spaced intervals during rounds $[K + 1, T]$ for 3 simulations with $T = 10^4$. Unlike Epsilon-Greedy, we observe that the UCB simulations tend to converge to the optimal policy and remain there. One potential explanation for this is that in UCB the exploration is dependent on the bonus terms for each arm, whereas in Epsilon-Greedy an arm is selected using round-robin during exploration rounds. This could result in selecting a suboptimal arm during exploration, but then by chance drawing an unusually high reward that steers the algorithm away from the optimal policy.

5.4.2 Practical Limitations of UCB

The proposed UCB algorithm has several practical limitations. The main drawback is that due to the added linear term in the optimization, the objective is no longer log-concave. We observe that this results in longer runtime due to increased time needed for the optimization step. For example, consider the main UCB experiment in Section 5.4.1 and the main Epsilon-Greedy experiment in Section 5.3.1. Both experiments use 10 horizon values and 10 simulations per horizon, and the horizon interval for UCB is an order of magnitude smaller than the horizon interval for Epsilon-Greedy. Yet UCB takes approximately 17 minutes to finish whereas Epsilon-Greedy finishes in just under 8 minutes. We also observe that the optimization can fail in the regime where K is large and as a result UCB does not learn a reasonable policy. For example, consider again the experiment from Section 5.2.2 where we have $N = 2$ agents and $K = 100$ arms and each arm $j \in [K]$ has one agent with $\mu_{*,j}^* = 0.3$ and the other agent with $\mu_{*,j}^* = 0.5$. The optimal policy assigns uniform probability to each arm. However, we observe with $T = 10^3$ that UCB learns a policy which assigns $p_1 = 0.8$ to one arm, $p_2 = 0.2$ to a second arm, and $p_j = 0$ to the rest. This experiment also suffers from runtime issues: it takes 17 minutes to run a single simulation with $T = 10^3$; in contrast, for the same experiment with $N = 3$ agents and $K = 3$ arms, UCB takes just 2 seconds to run and learn an acceptable policy. It is evident that the proposed UCB algorithm, while theoretically sound, does not lend itself easily to a practical implementation. In fact, this issue was later identified and addressed by Jones et al. [6]. Their key insight is to modify the objective function such that it only uses NSW but still makes use of confidence bounds. To do so, they construct an upper bound matrix $\hat{U}^t$ for $\hat{\mu}$ which is used directly in the NSW function so that $p^t \leftarrow \arg \max_{p \in \Delta^K} \text{NSW}(p, \hat{U}^t)$. The optimization is

now over a log-concave function which allows the implementation remains stable and no longer suffer from excessive runtime issues. They also manage to prove that with a strategic choice of $\hat{U}^t$ the original regret bound from [5] is not compromised.

6 Future Directions

This report explored the MA-MAB variant proposed by Hossain et al. [5]. Future work could focus on several directions. First, we observed that the proposed Epsilon-Greedy algorithm can learn the optimal policy and later fall back to a suboptimal policy. Is there a choice of ϵ^t that can provably prevent such behavior or is this inherent to the algorithm? We also observe that UCB tends to converge to the optimal policy and remain there. Along the same lines, can we prove that UCB will converge and remain at the optimal solution? Answering these convergence questions could potentially provide guarantees that would be useful for deploying these algorithms in real-life policy decision situations. Also, the theoretical regret bounds for each algorithm were strong in the sense that they are very similar to their MAB counterparts. However, we observed that the proposed UCB algorithm does not offer a practical implementation, limiting its scope. Jones et al. [6] provide alternative UCB algorithms to address this, but ambitious future work could focus on introducing alternative algorithms for this MA-MAB variant which don't rely a continuous optimization step at all.

References

- [1] Mridul Agarwal, Vaneet Aggarwal, and Kamyar Azizzadenesheli. Multi-agent multi-armed bandits with limited communication, 2021.
- [2] Georgios Amanatidis, Georgios Birmpas, Aris Filos-Ratsikas, Alexandros Hollender, and Alexandros A. Voudouris. Maximum nash welfare and other stories about efx. *Theoretical Computer Science*, 863:69–85, April 2021.
- [3] Nima Anari, Tung Mai, Shayan Oveis Gharan, and Vijay V. Vazirani. Nash social welfare for indivisible items under separable, piecewise-linear concave utilities, 2017.
- [4] Ronshee Chawla, Daniel Vial, Sanjay Shakkottai, and R. Srikant. Collaborative multi-agent heterogeneous multi-armed bandits, 2024.
- [5] Safwan Hossain, Evi Micha, and Nisarg Shah. Fair algorithms for multi-agent multi-armed bandits. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 24005–24017. Curran Associates, Inc., 2021.
- [6] Matthew Jones, Huy Nguyen, and Thy Nguyen. An efficient algorithm for fair multi-agent multi-armed bandit with low regret. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(7):8159–8167, Jun. 2023.
- [7] Peter Landgren, Vaibhav Srivastava, and Naomi Ehrich Leonard. Distributed cooperative decision making in multi-agent multi-armed bandits. *Automatica*, 125:109445, 2021.